技术直播 · 拆开 Agent 框架的内核

Agent 框架祛魅

如果让你从零设计一个 Agent 框架，你会怎么决策？

LangGraph · Google ADK · CrewAI · Claude Code · ProdAgent 同台拆解

李号双 · eBay 资深架构师

配套开源 ProdAgent · 37 讲专栏

会写一个 Agent，不等于会设计 Agent 框架

一个能跑的 Agent，六行就够：

```
messages=[user]
while True:
r = llm.chat(messages)
if not r.tool_calls:
return r.text
messages += run_tools(r)
```

→

但企业要的，是一个能托住业务的底座

- 很多种业务流程，都要能在上面搭
- 成千上万人同时用，各走各的进度
- 要长期跑：崩了能续、能审计、能控成本
- 还得能停下等人审批、接上外部系统

所以今天不聊怎么调模型，只回答一个问题：

让你来造这个底座，动手前得先想清楚哪几件事？

写第一行代码前，架构师先定六件事

01	业务流程怎么表达，才好复用、好改、看得清?	第①站 · 流程结构
02	谁来推动执行，又不被具体业务绑死?	第②站 · 调度推进
03	一条流程万人同时跑，进度怎么互不干扰?	第③站 · 运行状态
04	中途要等人审批、等外部结果，怎么停怎么续?	第④站 · 暂停等人
05	监控、审计、预算、权限，怎么不塞进核心?	第⑤站 · 总线横切
06	崩了怎么不丢、不重、还能往回追溯?	第⑥站 · 持久恢复

六件不是凑数：少一样就缺一块能力；各种新框架，说到底都是这六样的不同组合，跟用哪家模型没关系。一站一件，往下看。

流程怎么表达：先写句法，还是现场造句

退款例子 · 查单和审核怎么连

路线 A · 先写句法：跑之前就定完

LangGraph · Google ADK · CrewAI Flows

步骤、连线、岔路开工前就画成一张图；模型只能在留好的岔路上选、在预留模板上多分几份，加不出图上没有的新步骤。

像铁路网：道岔铺好，司机只在道岔里选线。

得到：全局走向一眼可见，汇聚、恢复、换人续跑都好做。

代价：得先想清骨架，特别开放、画不全的任务会吃力。

路线 B · 现场造句：跑起来再组织

Claude Code

只给工具和子 Agent 这些“单词”；先后、拆几个、派谁，由模型读着对话和任务板当场组织。

像越野向导：只给装备不给图，路线当场踩。

得到：开放、探索型任务特别灵活，不必事先画全。

代价：全局没人总览，汇聚、续跑都更难——开放的对价。

这一站还没收敛：流程能预先想清、要审计要确定，用图；任务开放、探索性强，用循环——不同场景，不同选择。

谁来推进：一轮算齐，还是模型边走边定

退款例子 · 风控三项同时核查

按“波次”推进：一轮算齐

LangGraph · ADK（波次）· CrewAI（顺序）

每轮先看一遍：哪些步骤前面的活儿都齐了、能开工，把它们凑成一拨同时跑，结果一起收，再看下一拨。

退款：三项风控同轮并发，都回来再汇总。

得到：谁先跑完不影响结果、好汇总，每一拨收尾都是一个整齐的存档点。
代价：要等本轮最慢的那个。

模型驱动的循环 · 来一步算一步

Claude Code：一个 while 循环，模型决定下一步

没有独立调度器；模型每轮读着对话和任务板，自己决定调哪个工具、派给谁、还是收工，控制权在模型手里。

退款：像老练客服边走边判断，不是照一张排好的工序图施工。

得到：开放任务最灵活，临时加步骤、换路子都行。
代价：没人统管全局，想同时跑再汇总、想稳稳恢复，都得自己在外层补。

要结果稳、好汇总、好恢复，就“一拨一拨”来；要开放、灵活、边走边定，就让模型驱动循环——不同任务，选不同引擎。

一张流程万人跑，进度怎么互不串

退款例子 · 两笔退款同时来

统一做法 · 一次执行一个隔离容器

一次执行 = 一个带 id 的隔离容器，进度按 id 分区存放

流程本身不带状态、万人共享一张蓝图；每来一个请求，
就开一个带 id 的独立容器，各装各的进度。

退款：十个用户同时申请，就是十个容器，天然不串。

蓝图不带状态才能被无数次复用；进度一人一份，也是后面暂停、恢复的载体。

✓ 这一点已经收敛

没有哪家正经框架让一万人共享一份可变进度。LangGraph 的 thread、ADK 的 Session、CrewAI 的 flow state、Claude Code 的 session、ProdAgent 的 Run，是同一个隔离容器——一人一个 id，这件事五家没有分歧。

万人不串的钥匙：蓝图共享、进度私有——一次执行一个 id，状态按 id 隔开。

要等人时：所有 Agent 框架收敛到同一个动作

退款例子 · 大额退款等主管批

统一做法 · 暂停 = 落盘放手，不是占着位子死等

人在回路（HITL）：审批、补信息、等外部给结果

走到要等人的地方，先把“走到哪、在等什么”存成一份数据，然后把手里的活儿彻底放下、不占着位子；人可能几小时甚至几天后才回复，这期间服务器重启、加机器都不影响；值一到，从断点原样接着跑。

退款：大额退款卡在主管审批，等待的几小时里服务可以随便重启、换人，不用从头再来。

人慢、机器快，你不可能为一句“等审批”把位子占几个小时；这正好用得上第 3 站那份能存起来的进度。

✓ 这一点已经收敛

LangGraph 用 interrupt、Google ADK 用工具确认与长任务续跑、CrewAI 用人工输入，都先存好再挂起；Claude Code 的权限门也是没批准绝不执行、会话能接着聊。没有哪个正经框架会傻占着位子干等。

审批不是“等一会儿”，是“把现场交出去、再接回来”；图派和循环派在这件事上做法一致。

横切能力怎么进：挂在事件钩子上

退款例子 · 每步留痕、超预算拦截

统一做法 · 监控、审批、预算，全是挂上来的旁观者

观测 / 审计 / 预算 / 审批，都不改主干

内核每干完一步，就往外喊一声“我干了啥”；监控、记账、预算、审批像围观的人挂在旁边，想看就看、该拦就拦，随时能加能拆、互不影响，却谁都改不了主流程。

退款：每一步留痕、超预算自动拦截，退款主逻辑一行都不用动。

这类旁边的需求会越来越多，挂在外面就能随时加拆；要是写进主流程，每加一个都得动内核。

✓ 这一点已经收敛

LangGraph 用回调与中间件、Google ADK 用 plugin、CrewAI 用 hooks 与护栏、Claude Code 用 hooks 与权限门——样子不同、位置一致：旁观者只看、能拦，但绝不替主流程做决定。

横切能力是“外挂”不是“内嵌”：内核只负责把事件发出去，剩下的交给旁观者。

崩了怎么不丢不重：只追加事实，状态随时重推

退款例子 · 崩在批准后不重复打款

收敛方向 · 只追加发生过的事实，当前状态随时重推

都在显式留痕：LangGraph、CrewAI 存快照，ADK 这类追加事件

每发生一件事，就往小本子上补记一笔、记完不改；“现在到哪了”不用另存，把记录从头捋一遍就知道，快照只是为了省时间、免得每次从头捋。

恢复时先读最近的快照，再把之后几笔补上；连“钱已经打了”这种结果也记着，重来时不会把同一笔退款再打一次。

不丢、不重、每步都能查，还能回到任意一步换条路；快照和事件是两个深浅档次，越把“发生过的事实”当根本，恢复和“回到过去”越省事。

✓ 方向已经收敛

要可靠地办“打款、下单”这种一做就有后果的事，光存聊天记录不够；行业都在往“把事实记清楚、状态能随时重算”走，差别只在做多彻底。

别把“现在到哪了”当唯一的存档；发生过的事实才是唯一真相，当前状态不过是从这些事实里捋出来的结果。

同台对照 · 建议截图

五家放在同一张尺子下

名字千差万别，填进同一张表

框架	流程结构	调度推进	运行状态	暂停等人	横切挂载	持久恢复
LangGraph	显式图、可环	超步算就绪	Channel+合并	interrupt()	回调/中间件	超步状态快照
Google ADK	新版图引擎，Agent 也是节点	图就绪 + 屏障重放	Session事件折叠	工具确认/续跑	回调 / plugin	事件持久·重建
CrewAI	角色任务·Flows	顺序/主管派活	Flow结构化state	人工输入	Hooks/护栏	Flows快照续跑
Claude Code	agentic循环·任务板	模型驱动 while	session·任务板	权限门·可续	hooks·权限门	会话持久·resume
ProdAgent	节点边·Goto/Send	就绪集合+波次	Run+通道+Run树	节点边界放手	Bus三协议	事件为事实·重放

LangGraph　生态最全、控制力最强，概念也最重

Google ADK　新版底层已是图引擎、Agent 可作节点；Session 仍是一串事件

CrewAI　上手最快；Flows 能快照续跑，memory 管的是记忆

Claude Code　最自主的循环派代表：会话持久化加权限门，单机交互最顺手。

ProdAgent　约1800行、零依赖，把“事件为事实”做到最纯粹，可周末读完复刻

同一把尺子：没有谁更高级，只有为不同目标做的取舍。

而且两派正在会师：图派把自主 Agent 收编成节点，循环派在循环外包确定性骨架。

答案揭晓 · ProdAgent 的选择

同一幅图 · 六格填满

六个格子收敛成六个部件，拼成一台内核



六个问题，六个各管一件、互不重叠的部件，不多不少——这就是 ProdAgent 的内核。

机制在内核，ReAct、规划、多 Agent 都在外头拼，接下来就拼给你看。

那些熟悉的范式，都是同一把零件拼出来的

ReAct

内核里没有 ReAct 这个类。
它就是“想 → 做 → 想”三个节点，工具做完用一条回边把“思考”重新点亮。

退款：判断 → 查单 → 再判断 → 答复

内建 Agent 只是这张图的简写

plan-first

模型产出的“计划”，只是状态里一份可改的数据，不是另一张执行图。
写清单 → 动态分出多个工人 → 汇总等齐。

退款：拆成查单 / 核政策 / 风控并行

改计划 = 改数据，引擎不动

生成 — 审查（反思）

生成节点出初稿，审查节点挑毛病；不满意就沿回边重做，直到通过。

退款：建议初稿 → 合规审查 → 打回重写

同样不需要新引擎

连 Claude Code 式“边做边建任务”的循环，画到底也是同零件的图——内核一个零件不加。

多 Agent：抓住两个选择，五花八门的协作都能对上号

	信息：显式传递（带着上下文走）	信息：共享黑板（写同一块状态）
控制权 委派·去而返	**主管制 / 层级制** 主管派给专家一次子运行，专家干完把结果交回，主管始终在控。 supervisor / hierarchical	**并行专家 / 黑板** 一次派一批工人写同一块板、等齐回收；多轮再派由仲裁者收敛，便是黑板。 orchestrator–worker / blackboard
控制权 交棒·去不返	**交棒网络（蜂群）** 当前 Agent 判断该别人接，带着上下文一跳过去，不画回边、一去不返。 swarm / handoff · 账单交棒风控	**去中心化群聊（编舞）** 没有固定指挥，大家盯同一块共享状态，按规则或模型轮流上前，控制权不回到固定主管。 group chat / choreography

落到内核就是几个动作：调一次子运行、或扇出后用 join 收回（委派·去而返）；一次不回头的跳转（交棒）；往共享通道里写（黑板）。

换个视角还能分有没有“中心指挥”：有指挥叫编排 orchestration，大家自组织叫编舞 choreography。

让内核不认识任何一家厂商

内核只定“端口”：规定接口，不认实现

- 模型端口：进来是消息，出去是统一回复
- 工具端口：名字 + 说明书 + 一次执行
- 存储端口：写事件 / 重放重建
- 上下文装配：按 token 预算，现拼本次输入

内核只知道“有这么个能力”，不知道背后是哪家公司、哪个库。

外围全是可替换的策略

- 模型：任意厂商，或一个离线脚本（测试不花钱）
- 工具：本地函数 / MCP 服务，在边界归一成一种
- 上下文：截断、摘要、逐级压缩，随便换
- 长期记忆：关键词到向量库，只留两个方法
- 技能：按目录发现、按需加载

换实现，内核一行不改。

依赖方向永远朝内：外围依赖内核的接口，内核不依赖任何外围。

空口无凭 · 看得到、跑得完、读得完

这副骨架不大，你一个周末读得完

≈1800 行

内核只有 14 个文件，短到能从头读完

0 个三方依赖

运行时只用 Python 标准库，没有版本地狱

测试与十几个例子，全部离线跑、不花 token

它不是玩具。断点续跑、时间旅行、并发屏障、流式背压、人工审批、多 Agent，一样不少。

祛魅不是贬低。主流框架工程很强、生产可放心用；这门课只是带你从黑盒外面，走进内核。

37 讲三级台阶：搭内核 → 拼范式 → 接外部，每步都能自己动手复现。

---

github.com/limenagent/prodagent — 专栏每一讲，都对得上这里的代码

最后，想留给你一句话

框架每半年换一代，
看穿框架的能力，不过时。

把复杂系统剥到核心抽象，在未知里判断什么该稳定、什么该变化——这才是真正要练的东西。

---

github.com/limenagent/prodagent　现在就能去读、去跑，全程离线

《生产级 Agent 框架设计与实现》37 讲，带你从零搭一台属于自己的内核；只需基础 Python。

李号双 · eBay 资深架构师